Name: Rahin Ibne Harun

Id: 23-54177-3

Sec: U

Course: Compiler Design Assignment

Final Lab Task-4

Task

Compiler Design (Finalterm Lab 4)

Task 1: Write a C++ program that takes a **DFA transition table** as input and minimizes the DFA by removing equivalent states.

Your program must:

1. Read the DFA information from the user.
2. Separate states into final and non-final groups.
3. Check whether states in the same group behave equivalently for every input symbol.
4. Merge equivalent states.
5. Print the minimized DFA transition table.

Input Format

- The first line contains two integers n and m.
  - n = number of states, states are numbered from 0 to n-1
  - m = number of input symbols
- The next m lines contain the input symbols.
- The next n × m lines contain the DFA transition table in this format:
- current_state input_symbol next_state
- The next line contains the start state.
- The next line contains an integer f, the number of final states.
- The last line contains f integers representing the final states.

Output Format

Print:

- the groups of equivalent states
- the start state of the minimized DFA
- the final states of the minimized DFA

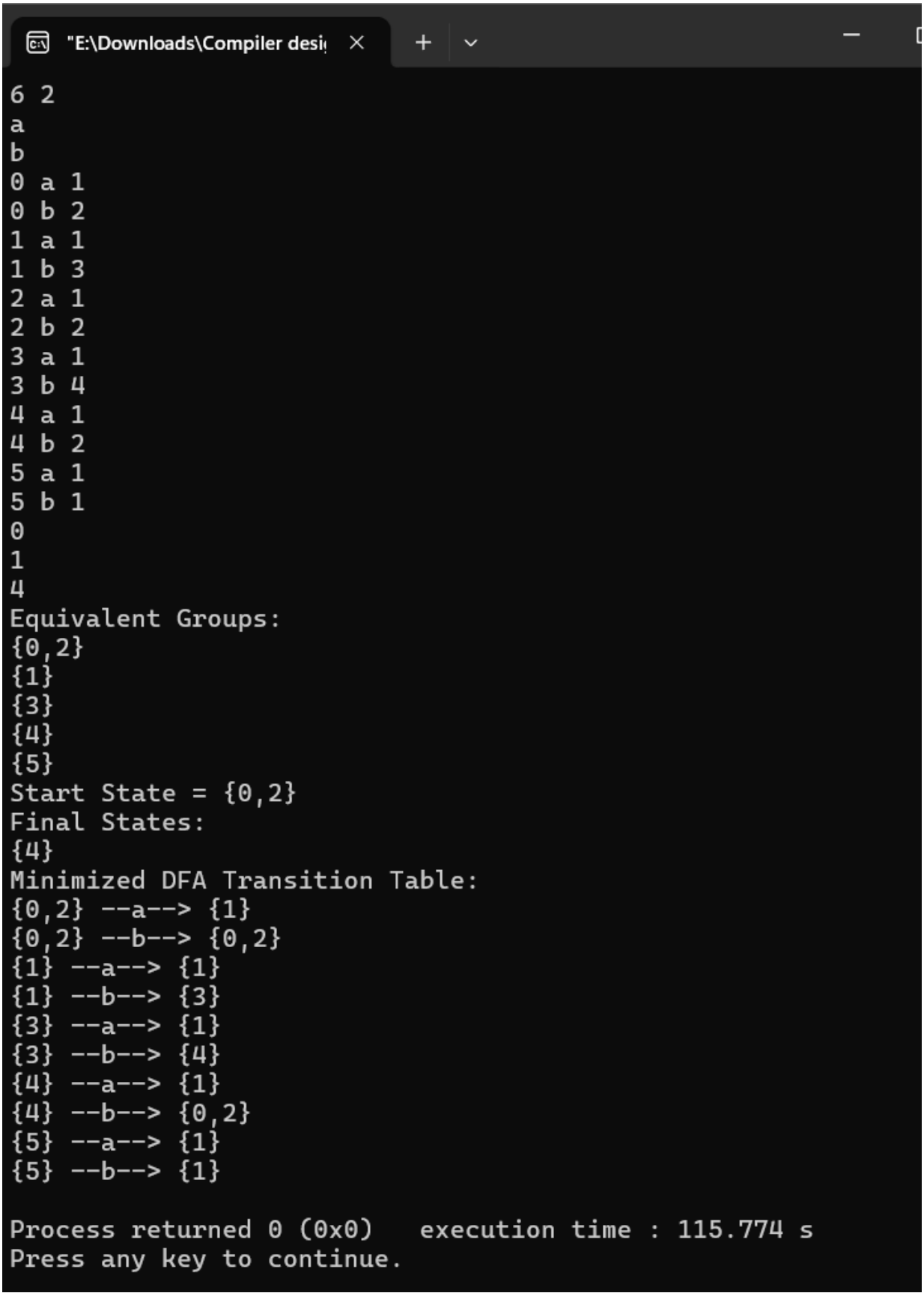
Ans:

```
#include <iostream>
#include <vector>
#include <string>
#include <set>
#include <map>
#include <algorithm>

using namespace std;

struct Transition {
   int next_state;
```

```
};

int main() {
    int n, m;
    if (!(cin >> n >> m)) return 0;

    vector<string> symbols(m);
    for (int i = 0; i < m; ++i) cin >> symbols[i];
    vector<vector<int>> adj(n, vector<int>(m));
    for (int i = 0; i < n * m; ++i) {
        int curr, next;
        string sym;
        cin >> curr >> sym >> next;
        for (int j = 0; j < m; ++j) {
            if (symbols[j] == sym) adj[curr][j] = next;
        }
    }

    int start_state;
    cin >> start_state;

    int f_count;
    cin >> f_count;
    vector<bool> is_final(n, false);
    for (int i = 0; i < f_count; ++i) {
        int f;
        cin >> f;
        is_final[f] = true;
    }
    vector<int> group(n);
    int num_groups = 0;
    if (f_count < n) num_groups++;
    if (f_count > 0) num_groups++;

    int non_final_id = 0, final_id = (f_count < n) ? 1 : 0;

    for (int i = 0; i < n; ++i) {
        group[i] = is_final[i] ? final_id : non_final_id;
    }
    while (true) {
```

```
    vector<int> new_group(n);
    map<pair<int, vector<int>>, int> partition_map;
    int next_id = 0;

    for (int i = 0; i < n; ++i) {
      vector<int> move_to;
      for (int j = 0; j < m; ++j) {
        move_to.push_back(group[adj[i][j]]);
      }

      pair<int, vector<int>> signature = {group[i], move_to};
      if (partition_map.find(signature) == partition_map.end()) {
        partition_map[signature] = next_id++;
      }
      new_group[i] = partition_map[signature];
    }

    if (next_id == num_groups) break;
    group = new_group;
    num_groups = next_id;
}
vector<vector<int>> equiv_groups(num_groups);
for (int i = 0; i < n; ++i) {
    equiv_groups[group[i]].push_back(i);
}

auto get_group_str = [&](int state_idx) {
    int g_id = group[state_idx];
    string s = "{";
    for (int i = 0; i < equiv_groups[g_id].size(); ++i) {
      s += to_string(equiv_groups[g_id][i]);
      if (i < (int)equiv_groups[g_id].size() - 1) s += ",";
    }
    s += "}";
    return s;
};

cout << "Equivalent Groups:" << endl;
for (auto& g : equiv_groups) {
    cout << "{";
```

```
        for (int i = 0; i < g.size(); ++i) {
            cout << g[i] << (i == g.size() - 1 ? "" : ",");
        }
        cout << "}" << endl;
    }

    cout << "Start State = " << get_group_str(start_state) << endl;

    cout << "Final States:" << endl;
    set<int> printed_finals;
    for (int i = 0; i < n; i++) {
        if (is_final[i] && printed_finals.find(group[i]) == printed_finals.end()) {
            cout << get_group_str(i) << " ";
            printed_finals.insert(group[i]);
        }
    }
    cout << endl;

    cout << "Minimized DFA Transition Table:" << endl;
    set<int> visited_groups;
    for (int i = 0; i < n; i++) {
        if (visited_groups.find(group[i]) == visited_groups.end()) {
            for (int j = 0; j < m; j++) {
                cout << get_group_str(i) << " --" << symbols[j] << "--> " << get_group_str(adj[i][j])
<< endl;
            }
            visited_groups.insert(group[i]);
        }
    }

    return 0;
}
output:
```


"E:\Downloads\Compiler desi
6 2
a
b
0 a 1
0 b 2
1 a 1
1 b 3
2 a 1
2 b 2
3 a 1
3 b 4
4 a 1
4 b 2
5 a 1
5 b 1
0
1
4
Equivalent Groups:
{0,2}
{1}
{3}
{4}
{5}
Start State = {0,2}
Final States:
{4}
Minimized DFA Transition Table:
{0,2} --a--> {1}
{0,2} --b--> {0,2}
{1} --a--> {1}
{1} --b--> {3}
{3} --a--> {1}
{3} --b--> {4}
{4} --a--> {1}
{4} --b--> {0,2}
{5} --a--> {1}
{5} --b--> {1}
Process returned 0 (0x0) execution time : 115.774 s
Press any key to continue.

Task2:

Task 2: Extend Task 1 so that after minimizing the DFA, the program takes multiple input strings and checks whether each string is **Accepted** or **Rejected** by the minimized DFA.

Your program must:

1. Read the DFA.
2. Minimize the DFA.
3. Print the minimized transition table.
4. Take multiple input strings.
5. For each string, print the state path and final result.

Input Format

Use the same DFA input format as Task 1, then add:

- one line containing integer q, the number of test strings
- next q lines, one input string per line

Output Format

For each input string, print:

- the input string
- the path through the minimized DFA
- Accepted or Rejected

Answer:

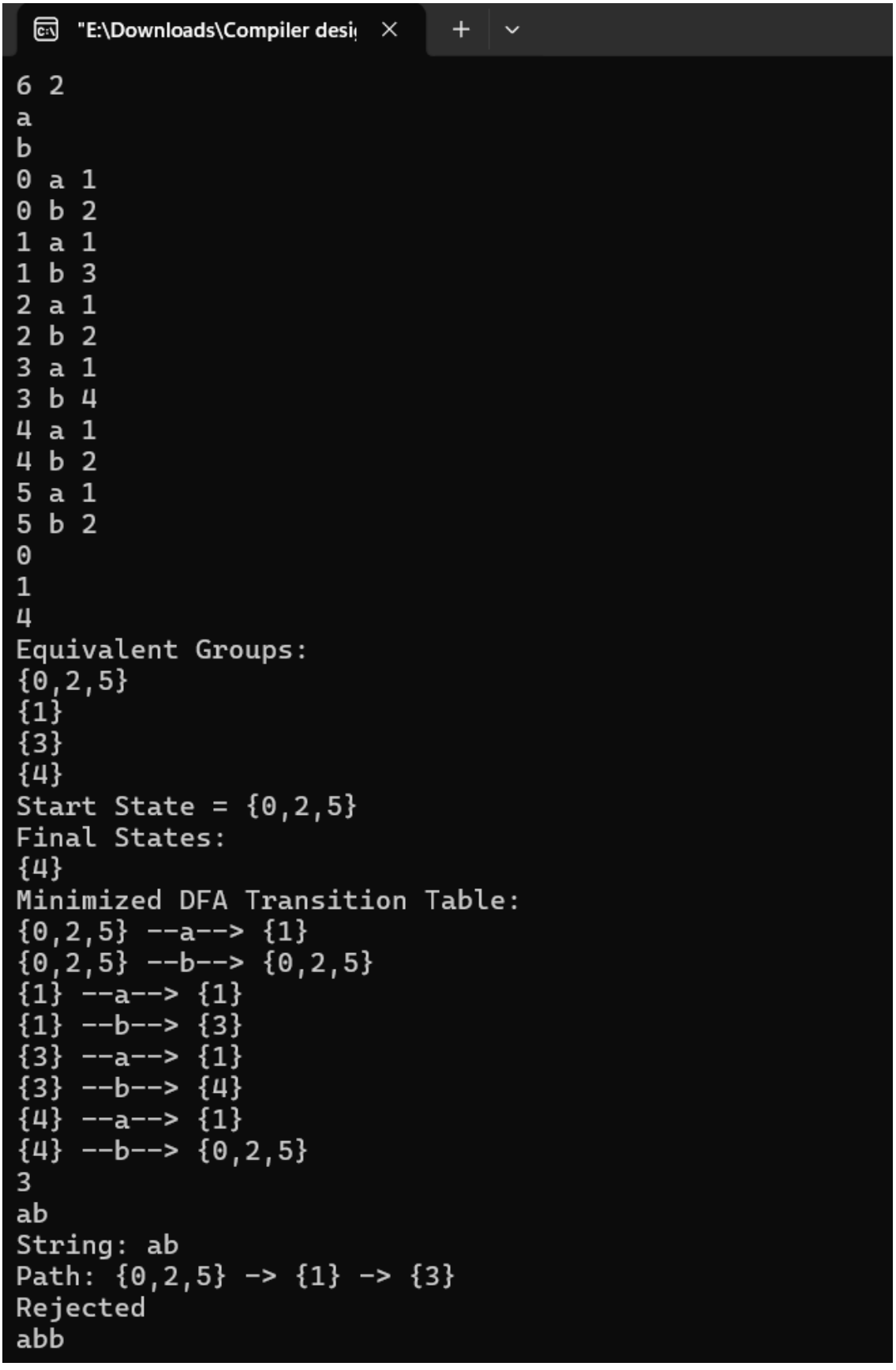
```
#include <iostream>
#include <vector>
#include <string>
#include <map>
#include <set>
#include <algorithm>

using namespace std;

string formatGroup(const vector<int>& states) {
   string s = "{";
   for (int i = 0; i < states.size(); ++i) {
      s += to_string(states[i]);
      if (i < (int)states.size() - 1) s += ",";
   }
   s += "}";
   return s;
}

int main() {
   int n, m;
   if (!(cin >> n >> m)) return 0;
```

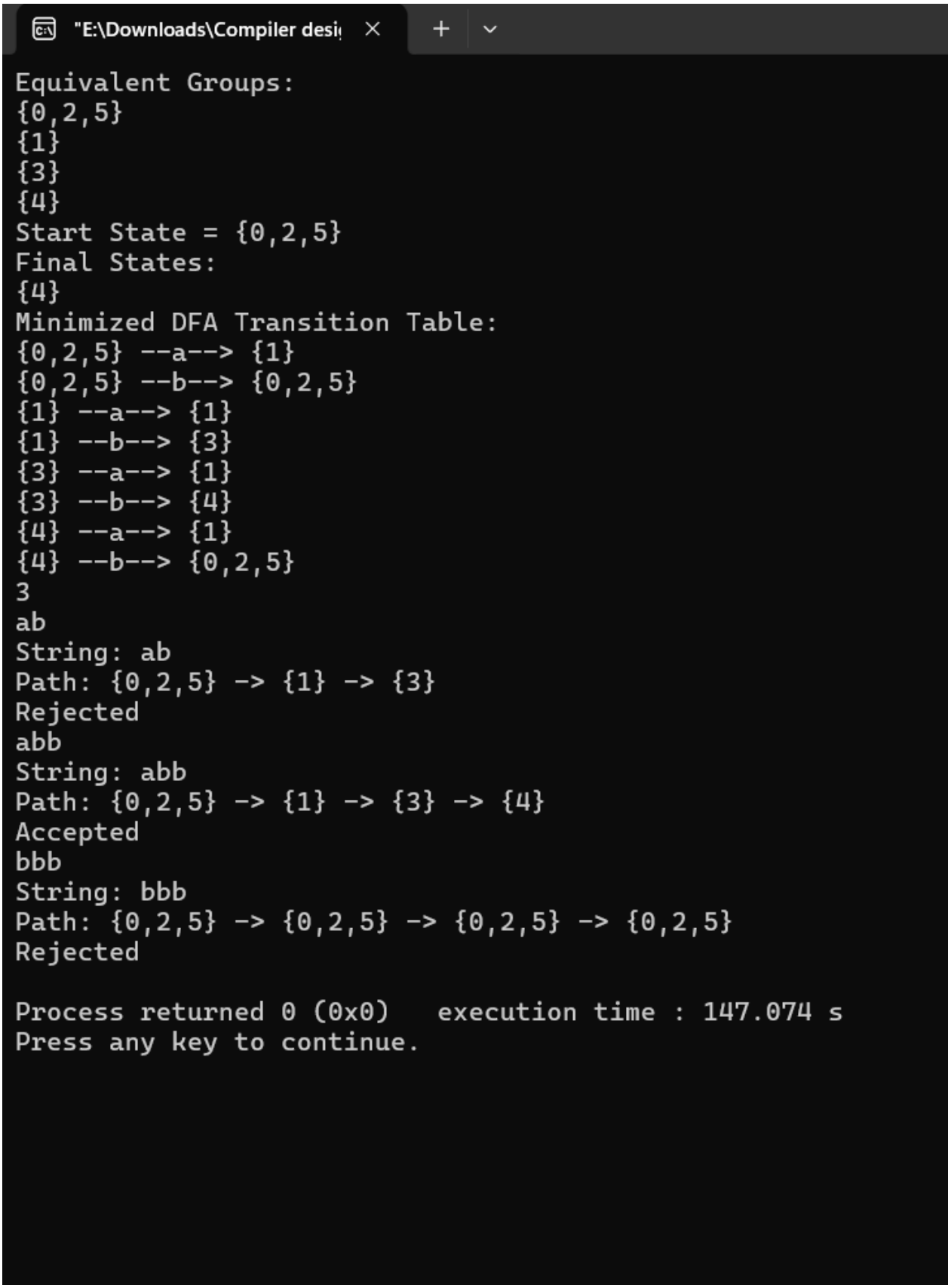
```
vector<string> symbols(m);
map<string, int> symIndex;
for (int i = 0; i < m; ++i) {
   cin >> symbols[i];
   symIndex[symbols[i]] = i;
}

vector<vector<int>> adj(n, vector<int>(m));
for (int i = 0; i < n * m; ++i) {
   int curr, next;
   string sym;
   cin >> curr >> sym >> next;
   adj[curr][symIndex[sym]] = next;
}

int start_node;
cin >> start_node;

int f_count;
cin >> f_count;
vector<bool> is_final(n, false);
for (int i = 0; i < f_count; ++i) {
   int f;
   cin >> f;
   is_final[f] = true;
}
vector<int> group(n);
int num_groups = 0;
if (f_count < n) num_groups++;
if (f_count > 0) num_groups++;
int non_final_id = 0, final_id = (f_count < n) ? 1 : 0;

for (int i = 0; i < n; ++i) group[i] = is_final[i] ? final_id : non_final_id;

while (true) {
   vector<int> new_group(n);
   map<pair<int, vector<int>>, int> partition_map;
   int next_id = 0;
   for (int i = 0; i < n; ++i) {
      vector<int> move_to;
      for (int j = 0; j < m; ++j) move_to.push_back(group[adj[i][j]]);
      pair<int, vector<int>> sig = {group[i], move_to};
      if (partition_map.find(sig) == partition_map.end()) partition_map[sig] = next_id++;
      new_group[i] = partition_map[sig];
   }
   if (next_id == num_groups) break;
   group = new_group;
```

```
    num_groups = next_id;
}
vector<vector<int>> equiv_groups(num_groups);
for (int i = 0; i < n; ++i) equiv_groups[group[i]].push_back(i);

vector<string> group_names(num_groups);
vector<bool> group_is_final(num_groups, false);
for (int i = 0; i < num_groups; i++) {
    group_names[i] = formatGroup(equiv_groups[i]);
    group_is_final[i] = is_final[equiv_groups[i][0]];
}
cout << "Equivalent Groups:" << endl;
for (int i = 0; i < num_groups; i++) cout << group_names[i] << endl;

cout << "Start State = " << group_names[group[start_node]] << endl;
cout << "Final States:" << endl;
for (int i = 0; i < num_groups; i++) if (group_is_final[i]) cout << group_names[i] << " ";
cout << "\nMinimized DFA Transition Table:" << endl;
vector<vector<int>> min_adj(num_groups, vector<int>(m));
for (int i = 0; i < num_groups; i++) {
    int representative = equiv_groups[i][0];
    for (int j = 0; j < m; j++) {
        min_adj[i][j] = group[adj[representative][j]];
        cout << group_names[i] << " --" << symbols[j] << "--> " << group_names[min_adj[i][j]] << endl;
    }
}
int q;
cin >> q;
while (q--) {
    string input;
    cin >> input;
    cout << "String: " << input << endl;
    cout << "Path: " << group_names[group[start_node]];

    int current_g = group[start_node];
    bool possible = true;

    for (char c : input) {
        string s(1, c);
        if (symIndex.find(s) == symIndex.end()) {
            possible = false;
            break;
        }
        current_g = min_adj[current_g][symIndex[s]];
        cout << " -> " << group_names[current_g];
    }

    if (possible && group_is_final[current_g]) cout << "\nAccepted" << endl;
```

```
        else cout << "\nRejected" << endl;
    }

    return 0;
}
```

Output:


"E:\Downloads\Compiler desi
6 2
a
b
0 a 1
0 b 2
1 a 1
1 b 3
2 a 1
2 b 2
3 a 1
3 b 4
4 a 1
4 b 2
5 a 1
5 b 2
0
1
4
Equivalent Groups:
{0,2,5}
{1}
{3}
{4}
Start State = {0,2,5}
Final States:
{4}
Minimized DFA Transition Table:
{0,2,5} --a--> {1}
{0,2,5} --b--> {0,2,5}
{1} --a--> {1}
{1} --b--> {3}
{3} --a--> {1}
{3} --b--> {4}
{4} --a--> {1}
{4} --b--> {0,2,5}
3
ab
String: ab
Path: {0,2,5} -> {1} -> {3}
Rejected
abb


"E:\Downloads\Compiler desi
Equivalent Groups:
{0,2,5}
{1}
{3}
{4}
Start State = {0,2,5}
Final States:
{4}
Minimized DFA Transition Table:
{0,2,5} --a--> {1}
{0,2,5} --b--> {0,2,5}
{1} --a--> {1}
{1} --b--> {3}
{3} --a--> {1}
{3} --b--> {4}
{4} --a--> {1}
{4} --b--> {0,2,5}
3
ab
String: ab
Path: {0,2,5} -> {1} -> {3}
Rejected
abb
String: abb
Path: {0,2,5} -> {1} -> {3} -> {4}
Accepted
bbb
String: bbb
Path: {0,2,5} -> {0,2,5} -> {0,2,5} -> {0,2,5}
Rejected
Process returned 0 (0x0) execution time : 147.074 s
Press any key to continue.

//////////////////////
End//////////////////////////////////